

Problem Regate

Autor problemă: STELIAN CHICHIRIM

Scurtă descriere a problemei

Se dă un graf neorientat cu N vârfuri și M muchii. Fiecare vârf i are asociată o valoare R_i , iar fiecare muchie j are un cost C_j . Definim o *regată* ca fiind un lanț de vârfuri în care, pentru oricare două vârfuri consecutive, muchia dintre ele este o punte în graful original. Pentru fiecare vârf, trebuie calculată suma valorilor tuturor regatelor care îl conțin.

Formal, pentru fiecare vârf v , trebuie calculată $\sum R_i$ pentru toate vârfurile i pentru care există o regată ce conține atât v , cât și i .

Observații cheie

Poduri și componente bicomponent

Observație 1. O punte într-un graf este o muchie a cărei eliminare crește numărul de componente conexe. Graful rămas după eliminarea tuturor punților constă din componente 2-conexe (bicomponente).

Observație 2. Orice regată trebuie să treacă exclusiv prin punți, deoarece o bicomponentă nu poate avea mai mult de o punte către restul grafului (altfel ar forma un ciclu).

Structura arborescentă a punților

Observație 3. Graful format din bicomponente (considerate ca noduri) și punți (considerate ca muchii) formează un arbore. Acest arbore este esențial pentru rezolvarea problemei.

Observație 4. Pentru fiecare vârf v , toate regatele care îl conțin sunt exact acele vârfuri aflate în componente biconexe situate în subarborele lui v în acest arbore al punților.

Soluție

Pasul 1: Identificarea punților

Folosim algoritmul lui Tarjan pentru găsirea punților în timp $O(N + M)$:

- Efectuăm o parcurgere DFS din rădăcină
- Pentru fiecare nod u , calculăm $lvl[u]$ (adâncimea în DFS) și $lvlMin[u]$ (nivelul minim accesibil din subarbore)
- O muchie (u, v) este punte dacă $lvlMin[v] > lvl[u]$

Pasul 2: Construirea componentelor biconexe

După identificarea punților, executăm un DFS care traversează doar muchiile care **nu** sunt punți. Fiecare componentă astfel obținută primește un identificator unic.

Pasul 3: Unirea componentelor în ordine descrescătoare a costurilor

- Construim lista tuturor punților, sortată după cost în ordine **descrescătoare**
- Pentru fiecare vârf i , adăugăm în listă o intrare specială $(-1, i, R_i)$ reprezentând valoarea asociată vârfului
- Parcurgem lista sortată:
 - Dacă $x = -1$, înseamnă că procesăm valoarea vârfului y : calculăm răspunsul pentru y folosind DSU curent
 - Altfel, unim componentele lui x și y folosind DSU cu euristică de dimensiune

Pasul 4: Propagarea valorilor

După ce toate componentele au fost unite într-o singură componentă, valorile acumulate în DSU trebuie propagate către toate nodurile. Efectuăm un DFS pe arborele format din operațiile de reuniune.

Corectitudine

Lemma 1

După pasul 2, fiecare bicomponentă formează exact o componentă conexă în graful în care am eliminat punțile.

Demonstrație. Prin definiția bicomponentelor, orice două vârfuri într-o bicomponentă sunt conectate prin cel puțin două căi intern-disjuncte. Eliminând punțile (care sunt muchii critice), aceste vârfuri rămân conectate. Conversely, vârfuri din bicomponente diferite nu pot fi conectate fără a folosi o punte, deoarece ar otherwise forma un ciclu. $\square$

Lemma 2

Ordinea procesării descrescătoare după cost asigură că, la momentul procesării unei valori R_i , toate punțile cu cost mai mare au fost deja procesate.

Demonstrație. Sorting-ul global asigură această proprietate direct. $\square$

Lemma 3

Pentru un vârf v procesat în pasul 3, componentele care conțin valori ce trebuie incluse în răspunsul său sunt exact cele deja unite în DSU.

Demonstrație. O valoare R_i contribuie la răspunsul lui v dacă și numai dacă există o regată de la v la i . Conform observației 2, aceasta înseamnă că i se află în subarborele lui v în arborele punților. Când procesăm o valoare, toate punțile din arbore care o conectează la v au fost deja procesate (Lemma 2), deci componentele sunt unite. $\square$

Complexitate

- Identificarea punților: $O(N + M)$
- DFS pentru componente: $O(N + M)$
- Sortarea punților: $O((N + M) \log(N + M))$
- Operațiile DSU: $O((N + M) \cdot \alpha(N + M)) \approx O(N + M)$

Complexitate totală: $O((N + M) \log(N + M))$

Implementare

Structuri de date principale

Structura pentru muchii și poduri:

```
struct edge {
    int x, c, pos;
};

struct bridge {
    int x, y, c;
};
```

Funcția de identificare a punților

```
void getBridges(int nod, int parent) {
    for (edge& vec : g[nod])
        if (lvl[vec.x] == 0) {
            lvl[vec.x] = lvlMin[vec.x] = lvl[nod] + 1;
            getBridges(vec.x, nod);
            if (lvlMin[vec.x] == lvl[vec.x]) {
                isBridge[vec.pos] = true;
                bridges.push_back({nod, vec.x, vec.c});
            }
            lvlMin[nod] = min(lvlMin[nod], lvlMin[vec.x]);
        }
}
```

```

        else if (vec.x != parent)
            lvlMin[nod] = min(lvlMin[nod], lvl[vec.x]);
    }

```

DSU cu reuniune prin dimensiune

```

void unite(int x, int y, int cst) {
    x = getRoot(x);
    y = getRoot(y);
    if (x == y) return;
    if (cnt[x] > cnt[y]) swap(x, y);
    v[y].push_back(x);
    val[y] += 1LL * cst * cnt[x];
    val[x] += 1LL * cst * cnt[y] - val[y];
    root[x] = y;
    cnt[y] += cnt[x];
}

```

Aici, *val* reprezintă diferența cumulativă între valoarea reală și cea calculată pentru fiecare componentă DSU. La reuniune, păstrăm invarianții necesari pentru propagarea corectă a valorilor.

Concluzii

Problema demonstrează eleganța combinării algoritmilor clasici (Tarjan pentru punți, DSU cu euristică) cu o manipulare atentă a valorilor. Ideea centrală este transformarea problemei grafice într-o structură arborescentă și apoi rezolvarea în ordine descrescătoare a costurilor, permițând calculul eficient al tuturor răspunsurilor în timp $O((N + M) \log(N + M))$.